



EMI305 · PRODUCCIÓN DE SOFTWARE · SISTEMAS CIBERFÍSICOS

Semana 4 — MDD4CPS: Desarrollo Dirigido por Modelos para CPS

La cadena de modelos · Tecnologías de transformación (XSLT, Python) · Correspondencias PIM → PSM · Grado de automatización · Herramientas.

Magíster en Ingeniería Informática · UFRO

EMI305 · Producción de Software · 2026



INSTITUCIÓN ACREDITADA
6 AÑOS
EN TODAS LAS ÁREAS
• GESTIÓN INSTITUCIONAL • DOCENCIA DE PREGRADO
• DOCENCIA DE POSTGRADO • INVESTIGACIÓN
• VINCULACIÓN CON EL MEDIO

Qué vamos a recorrer

01 La cadena de modelos CIM → PIM → PSM → Code

02 Tecnologías: XSLT, Python, diagrams.net

03 CIM → PIM: lo que se decide

04 PIM → PSM: lo que se decide

05 La fase Code

06 Correspondencias constructo → código

07 Grado de automatización (~78 %)

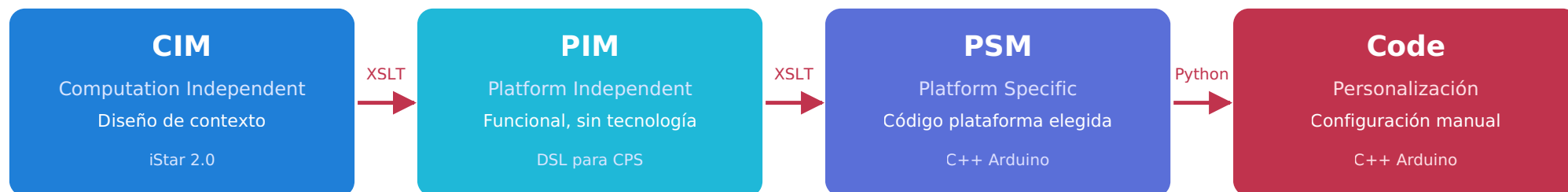
08 Herramientas por fase

09 Aplicación al caso SVIF

10 En síntesis · Referencias

MDD organiza el desarrollo como cadena de modelos

Cada transformación incorpora decisiones de diseño del siguiente nivel, preservando la información relevante del anterior.



Cada nivel agrega información no derivable del anterior; la trazabilidad se mantiene con identificadores (id_cim_parent).

Cómo se implementan las transformaciones

Opciones en el estado del arte:

ATL QVT ACCELEO EMF XML + XSLT

MDD4CPS optó por **tecnologías abiertas**:

- Modelado visual en **diagrams.net** → serializa a **XML**.
- Transformaciones con **XSLT** (CIM → PIM y PIM → PSM) y **Python** (PSM → Code).
- **Herramienta web** guía al diseñador en la captura de lo no derivable.

XSLT es declarativo: reconoce elementos del modelo de entrada y genera automáticamente la estructura correspondiente en el de salida — la misma idea de MDD. La etapa PIM → PSM reestructura el modelo (aún en XML) para dejarlo listo para la generación de código; el generador Python (`psm_to_code-arduinomkr1010.py`) es el que finalmente produce las fuentes Arduino.

¿QUÉ AGREGA EL DISEÑADOR?

Lo no derivable

CIM → **PIM** (aún independiente de tecnología):

- Periodicidad de las `OnIntervalAction`.
- Parámetros E/S de las `OnDemandAction`.
- Estructura del *dependum* y de los `SWResource`.

PIM → **PSM** (decisiones de plataforma):

- Plataforma objetivo (Arduino), MQTT, tipado, pines.

Qué genera cada constructo (PIM → PSM)

CONSTRUCTO PIM	SE MATERIALIZA COMO
CPCComponent	Unidad de despliegue: .ino + comm_utils.h + secrets.h
OnDemandAction	Función ejecutable con firma derivada de los parámetros del PIM
OnIntervalAction	Hilo periódico (FreeRTOS en Arduino MKR1010) cuyo período proviene de interval_in_milliseconds
MessageSender / MessageReceiver	Hilos de comunicación: publicar/suscribir, serializar/deserializar el <i>dependum</i>
SWResource	Estructuras de datos / variables tipadas
HWRResource	Comentarios estructurados y marcas de trazabilidad para integrar el hardware
Softgoals	No generan código ejecutable: comentarios de trazabilidad (Qualification/Contribution Array)
Refinamientos AND/OR	Estructuras lógicas (condicionales / funciones de evaluación)

¿Cuánto del código se genera automáticamente?

~78 %

del código final fue **generado** en el caso de estudio del curso (invernadero, Arduino MKR1010): **1216 de 1567 líneas**.

El ~22 % restante es configuración de plataforma y lógica propia.

FUENTE DEL DATO

Caso de estudio del curso

PROCESO

% GENERADO

MDD4CPS (Navarro et al., 2025) — invernadero

~78 %

Repositorio público del proceso: <https://github.com/mdd4cps/aomdd4cps>

Herramientas por fase:

CIM

PIM

→ diagrams.net

PSM

CODE

→ C++ Arduino

Toolchain de MDD4CPS

MODELADO VISUAL

diagrams.net

Editores gráficos + bibliotecas personalizadas (iStar 2.0, DSL PIM).

SERIALIZACIÓN

XML (mxGraph)

Los modelos se guardan como XML; susceptibles a transformaciones XML estándar.

TRANSFORMACIÓN

XSLT

CIM → PIM y PIM → PSM: declarativo, basado en reglas sobre el XML de diagrams.net.

GENERACIÓN

Python

PSM → Code: script que materializa hilos, funciones, structs y comentarios a partir del PSM.

Las transformaciones son **semiautomáticas y guiadas por el usuario**: la herramienta web solicita sólo los datos necesarios para el siguiente nivel.

SVIF recorre el ciclo completo

CIM

iStar 2.0 → 2 nodos agentes

Face Monitor (ESP32-CAM) y Access Actuator (ESP32) con objetivos, dependencias y softgoals. **26 elementos cim-***.

PIM

DSL CPS → arquitectura

2 CComponent, 2 OnIntervalAction (500 ms / 250 ms), 10 OnDemandAction, SW/HW Resources, Message Sender/Receiver.

PSM

C++ Arduino → ESP32 + MQTT

FaceMonitorComponent.ino + AccessActuatorComponent.ino + comm_utils.h + secrets.h.

CODE

Personalización

Credenciales, broker, umbral de confianza, política de autorización, SIMULATION_MODE.

~78 % del código generado · 14 eventos en la ejecución real (10 desbloques + 4 alarmas) · Rama del OR corregida tras hallazgo de verificación.

En síntesis

MDD4CPS concreta el MDD para CPS con la cadena **iStar** → **DSL** → **C++ Arduino** → **código final**.

Las transformaciones (**XSLT/Python**) capturan automáticamente lo trazable (*ids, nombres, relaciones, softgoals*) y piden al diseñador solo lo nuevo de cada nivel (períodos, *dependum*, plataforma, tipos).

Resultado: **~78 %** del código generado y **trazabilidad completa** desde el objetivo de negocio hasta el hilo que lo ejecuta.

FORTALEZA

Trazabilidad

softgoal → código.

FORTALEZA

Decisiones

postergadas al nivel correcto.

LIMITACIÓN

Restricciones

temporales duras no expresables en el DSL.

Referencias: Bézivin (2005); Navarro, Devia, Labra Gayo & Cares (2025).